

SKILL OSSP W8(S15&16)

SESSION 15:

TASK1: To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define INPUT_SIZE 1024
#define MAX_ARGS 64

char *trim(char *str)
{
    while (*str == ' ' || *str == '\t')
        str++;

    if (*str == '\0')
        return str;

    char *end = str + strlen(str) - 1;

    while (end > str &&
           (*end == ' ' || *end == '\t'))
        end--;

    *(end + 1) = '\0';

    return str;
}

int tokenize(char *command, char **args)
{
    int count = 0;

    char *token = strtok(command, " \t");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }

    args[count] = NULL;
}
```

```
GNU nano 7.2
return count;
}

int main()
{
    char input[INPUT_SIZE];

    printf("Combined Redirection Manager - Session 15 Task 1\n");
    printf("Use: command > output 2> error\n");
    printf("Use: command > output 2>&1\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("combined-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char *out_redirect = strstr(input, ">");
        char *err_redirect = strstr(input, "2>");

        if (out_redirect == NULL)
        {
            printf("Error: Output redirection '>' not found.\n");
            continue;
        }

        *out_redirect = '\0';

        char *command = trim(input);
        char *output_file = NULL;
        char *error_file = NULL;
    }
}
```

```
GNU nano 7.2

while (*remaining == ' ' || *remaining == '\t')
    remaining++;

char *error_pos = strstr(remaining, "2>");

if (error_pos != NULL)
{
    *error_pos = '\0';

    output_file = trim(remaining);
    error_file = trim(error_pos + 2);
}
else
{
    output_file = trim(remaining);
}

if (strlen(command) == 0 ||
    strlen(output_file) == 0)
{
    printf("Error: Invalid redirection syntax.\n");
    continue;
}

char command_copy[INPUT_SIZE];

strncpy(command_copy, command, INPUT_SIZE - 1);
command_copy[INPUT_SIZE - 1] = '\0';

char *args[MAX_ARGS];

if (tokenize(command_copy, args) == 0)
{
    printf("Error: Empty command.\n");
    continue;
}

int output_fd = open(
    output_file,
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);
```

```
GNU nano 7.2
if (output_fd == -1)
{
    perror("open output");
    continue;
}

int error_fd = -1;

if (error_file != NULL &&
    strlen(error_file) > 0)
{
    error_fd = open(
        error_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (error_fd == -1)
    {
        perror("open error");
        close(output_fd);
        continue;
    }
}

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");

    close(output_fd);

    if (error_fd != -1)
        close(error_fd);

    continue;
}

if (pid == 0)
{
    if (dup2(output_fd, STDOUT_FILENO) == -1)
```

```

koumudi@Koumudi: ~/OSSP/
GNU nano 7.2
if (pid == 0)
{
    if (dup2(output_fd, STDOUT_FILENO) == -1)
    {
        perror("dup2 stdout");
        exit(EXIT_FAILURE);
    }

    if (error_fd != -1)
    {
        if (dup2(error_fd, STDERR_FILENO) == -1)
        {
            perror("dup2 stderr");
            exit(EXIT_FAILURE);
        }
    }

    close(output_fd);

    if (error_fd != -1)
        close(error_fd);

    execvp(args[0], args);

    perror("execvp");
    exit(EXIT_FAILURE);
}

close(output_fd);
if (error_fd != -1)
    close(error_fd);

int status;
waitpid(pid, &status, 0);

if (WIFEXITED(status) &&
    WEXITSTATUS(status) == 0)
{
    printf("Command completed successfully.\n");
}
else
{
    printf("Command completed with errors.\n");
}

printf("Output saved to: %s\n", output_file);

if (error_file != NULL)
    printf("Errors saved to: %s\n", error_file);

printf("\n");
}

printf("Combined redirection manager terminated.\n");

return 0;
}

```

```

int status;

waitpid(pid, &status, 0);

if (WIFEXITED(status) &&
    WEXITSTATUS(status) == 0)
{
    printf("Command completed successfully.\n");
}
else
{
    printf("Command completed with errors.\n");
}

printf("Output saved to: %s\n", output_file);

if (error_file != NULL)
    printf("Errors saved to: %s\n", error_file);

printf("\n");
}

printf("Combined redirection manager terminated.\n");

return 0;
}

```

```

koumudi@Koumudi: ~/OSSP/
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ nano combined_redirect.c
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ gcc combined_redirect.c -o combined_redirect
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ ls
combined_redirect  combined_redirect.c
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ ./combined_redirect
Combined Redirection Manager - Session 15 Task 1
Use: command > output 2> error
Use: command > output 2>&1
Type 'exit' to quit.

combined-shell> echo Hello OSSP > output.txt 2> error.txt
Command completed successfully.
Output saved to: output.txt
Errors saved to: error.txt

combined-shell> exit
Combined redirection manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ cat output.txt
Hello OSSP
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ cat error.txt
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ ./combined_redirect
Combined Redirection Manager - Session 15 Task 1
Use: command > output 2> error
Use: command > output 2>&1
Type 'exit' to quit.

combined-shell> ls /wrong_directory > output2.txt 2> error2.txt
Command completed with errors.
Output saved to: output2.txt
Errors saved to: error2.txt

combined-shell> exit
Combined redirection manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_15$ cat error2.txt
ls: cannot access '/wrong_directory': No such file or directory
koumudi@Koumudi:~/OSSP/SKILL/Session_15$

```

TASK2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define INPUT_SIZE 1024
#define MAX_COMMANDS 6
#define MAX_ARGS 64

char *trim(char *str)
{
    while (*str == ' ' || *str == '\t')
        str++;

    char *end = str + strlen(str) - 1;
    while (end >= str &&
           (*end == ' ' || *end == '\t'))
    {
        *end = '\0';
        end--;
    }
    return str;
}

int tokenize(char *command, char **args)
{
    int count = 0;
    char *token = strtok(command, " \t");
    while (token != NULL &&
           count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;
}
```

```
GNU nano 7.2
char input[INPUT_SIZE];

printf("Complex Command Shell - Session 15 Task 2\n");
printf("Supports pipes and output redirection.\n");
printf("Example: ls | grep .c > result.txt\n");
printf("Type 'exit' to quit.\n\n");

while (1)
{
    printf("complex-shell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strlen(input) == 0)
        continue;

    char *output_redirect = strstr(input, ">");
    char *output_file = NULL;

    if (output_redirect != NULL)
    {
        *output_redirect = '\0';
        output_file = trim(output_redirect + 1);

        if (strlen(output_file) == 0)
        {
            printf("Error: Output file missing.\n");
            continue;
        }
    }

    char *commands[MAX_COMMANDS];
    int command_count = 0;
```

```
GNU nano 7.2
char *token = strtok(input, "|");

while (token != NULL &&
       command_count < MAX_COMMANDS)
{
    commands[command_count++] = trim(token);
    token = strtok(NULL, "|");
}

if (command_count == 0)
{
    printf("Error: No command found.\n");
    continue;
}

printf("\nExecution plan:\n");
for (int i = 0; i < command_count; i++)
{
    printf("Command %d: %s\n",
           i + 1,
           commands[i]);
}

if (output_file != NULL)
{
    printf("Final output -> %s\n",
           output_file);
}

printf("\n");
int pipes[MAX_COMMANDS - 1][2];
for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe");
        continue;
    }
}
```

```
GNU nano 7.2
        exit(EXIT_FAILURE);
    }
    dup2(fd, STDOUT_FILENO);
    close(fd);
}

for (int j = 0;
     j < command_count - 1;
     j++)
{
    close(pipes[j][0]);
    close(pipes[j][1]);
}

execvp(args[0], args);

perror("execvp");
exit(EXIT_FAILURE);
}

for (int i = 0;
     i < command_count - 1;
     i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("Pipeline execution completed.\n\n");

printf("Complex command shell terminated.\n");

return 0;
}
```

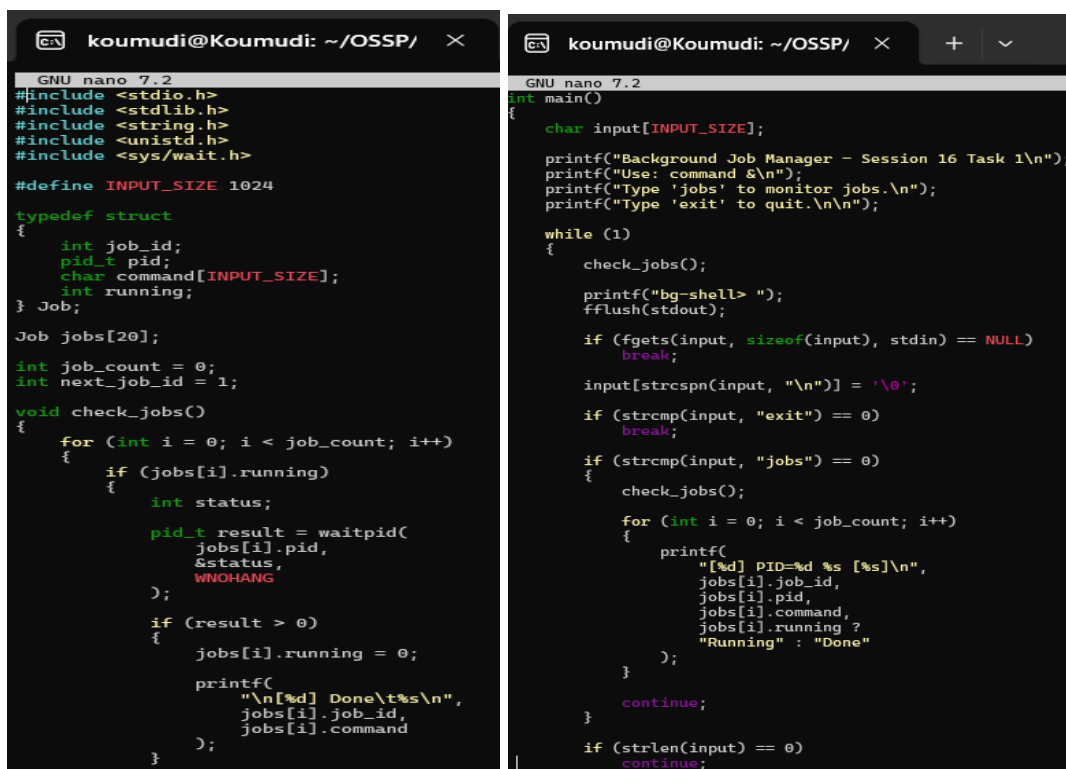
```

koumudi@Koumudi: ~/OSSP/SKILL/Session_15$ nano complex_shell.c
koumudi@Koumudi: ~/OSSP/SKILL/Session_15$ gcc complex_shell.c -o complex_shell
koumudi@Koumudi: ~/OSSP/SKILL/Session_15$ ls | grep .c > result.txt
koumudi@Koumudi: ~/OSSP/SKILL/Session_15$ cat result.txt
combined_redirect
combined_redirect.c
complex_shell.c
koumudi@Koumudi: ~/OSSP/SKILL/Session_15$

```

SESSION 16:

TASK1: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.



```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define INPUT_SIZE 1024

typedef struct
{
    int job_id;
    pid_t pid;
    char command[INPUT_SIZE];
    int running;
} Job;

Job jobs[20];
int job_count = 0;
int next_job_id = 1;

void check_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;
            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );
            if (result > 0)
            {
                jobs[i].running = 0;
                printf(
                    "\n[%d] Done\t%s\n",
                    jobs[i].job_id,
                    jobs[i].command
                );
            }
        }
    }
}

int main()
{
    char input[INPUT_SIZE];

    printf("Background Job Manager - Session 16 Task 1\n");
    printf("Use: command &\n");
    printf("Type 'jobs' to monitor jobs.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        check_jobs();
        printf("bg-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0)
        {
            check_jobs();
            for (int i = 0; i < job_count; i++)
            {
                printf(
                    "[%d] PID=%d %s [%s]\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].command,
                    jobs[i].running ?
                    "Running" : "Done"
                );
            }
            continue;
        }

        if (strlen(input) == 0)
            continue;
    }
}

```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
if (strlen(input) == 0)
    continue;

int background = 0;

if (input[strlen(input) - 1] == '&')
{
    background = 1;
    input[strlen(input) - 1] = '\\0';
    while (strlen(input) > 0 &&
           input[strlen(input) - 1] == ' ')
    {
        input[strlen(input) - 1] = '\\0';
    }
}

pid_t pid = fork();
if (pid == -1)
{
    perror("fork");
    continue;
}

if (pid == 0)
{
    execl("/bin/sh",
          "sh",
          "-c",
          input,
          (char *)NULL);

    perror("exec");
    exit(EXIT_FAILURE);
}

if (background)
{
    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    strncpy(
        jobs[job_count].command,
        input,
        INPUT_SIZE - 1
    );
    jobs[job_count].command[INPUT_SIZE - 1] = '\\0';
    jobs[job_count].running = 1;
    job_count++;
    printf(
        "[%d] %d\\n",
        jobs[job_count - 1].job_id,
        pid
    );
    printf(
        "Background job started. "
        "Prompt returned immediately.\\n"
    );
}
else
{
    waitpid(pid, NULL, 0);
    printf("Foreground command completed.\\n");
}
}

printf("Background job manager terminated.\\n");
return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
koumudi@Koumudi:~$ cd ~/OSSP/SKILL
mkdir -p Session_16
cd Session_16
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ nano background_shell.c
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ gcc background_shell.c -o background_shell
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ ./background_shell
Background Job Manager - Session 16 Task 1
Use: command &
Type 'jobs' to monitor jobs.
Type 'exit' to quit.

bg-shell> sleep 10 &
[1] 881
Background job started. Prompt returned immediately.
bg-shell> jobs

[1] Done          sleep 10
[1] PID=881 sleep 10 [Done]
bg-shell> jobs
[1] PID=881 sleep 10 [Done]
bg-shell> sleep 5 &
[2] 885
Background job started. Prompt returned immediately.
bg-shell> exit
Background job manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ |
```

TASK2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 20
#define INPUT_SIZE 1024

typedef enum
{
    JOB_RUNNING,
    JOB_DONE
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[INPUT_SIZE];
    JobState state;
} Job;

Job job_table[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].state == JOB_RUNNING)
        {
            int status;

            pid_t result = waitpid(
                job_table[i].pid,
                &status,
                WNOHANG
            );
        }
    }
}

```

```

GNU nano 7.2
        if (result > 0)
        {
            job_table[i].state = JOB_DONE;
        }
    }
}

void show_jobs()
{
    update_jobs();

    printf("\nJob Table\n");
    printf(
        "-----\n"
    );
    printf(
        "ID\tPID\tPGID\tSTATE\tCOMMAND\n"
    );
    printf(
        "-----\n"
    );
    for (int i = 0; i < job_count; i++)
    {
        printf(
            "%d\t%d\t%d\t%s\t%s\n",
            job_table[i].job_id,
            job_table[i].pid,
            job_table[i].pgid,
            job_table[i].state ==
            JOB_RUNNING ?
            "Running" : "Done",
            job_table[i].command
        );
    }
    printf(
        "-----\n"
    );
}

```

```

GNU nano 7.2
void remove_completed_jobs()
{
    int write_index = 0;

    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].state == JOB_RUNNING)
        {
            job_table[write_index++] =
                job_table[i];
        }
    }
    job_count = write_index;
}

int main()
{
    char input[INPUT_SIZE];

    printf("Job Table Manager - Session 16 Task 2\n");
    printf("Use: command &\n");
    printf("Commands: jobs, cleanup, exit\n");

    while (1)
    {
        update_jobs();

        printf("job-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0)
        {

```

```

        printf(
            "Completed jobs removed from job table.\n"
        );
        continue;

        if (strlen(input) == 0)
            continue;

        int background = 0;

        if (input[strlen(input) - 1] == '&')
        {
            background = 1;
            input[strlen(input) - 1] = '\0';

            while (strlen(input) > 0 &&
                input[strlen(input) - 1] == ' ')
            {
                input[strlen(input) - 1] = '\0';
            }
        }

        if (!background)
        {
            printf(
                "Error: Use '&' for background jobs.\n"
            );
            continue;
        }

        pid_t pid = fork();

        if (pid == -1)
        {
            perror("fork");
            continue;
        }
    }
}

```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2

    if (job_count < MAX_JOBS)
    {
        job_table[job_count].job_id =
            next_job_id++;

        job_table[job_count].pid = pid;

        job_table[job_count].pgid = pid;

        strncpy(
            job_table[job_count].command,
            input,
            INPUT_SIZE - 1
        );

        job_table[job_count]
            .command[INPUT_SIZE - 1] = '\0';

        job_table[job_count].state =
            JOB_RUNNING;

        printf(
            "[%d] PID=%d PGID=%d Running\n",
            job_table[job_count].job_id,
            pid,
            pid
        );

        job_count++;
    }
    else
    {
        printf("Job table is full.\n");
    }

    printf("Job table manager terminated.\n");

    return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾

bg-shell> exit
Background job manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ nano job_table.c
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ gcc job_table.c -o job_table
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ ./job_table
Job Table Manager - Session 16 Task 2
Use: command &
Commands: jobs, cleanup, exit

job-shell> sleep 10 &
[1] PID=969 PGID=969 Running
job-shell> jobs

Job Table
-----
ID      PID      PGID      STATE      COMMAND
-----
1        969      969      Running    sleep 10
-----

job-shell> jobs

Job Table
-----
ID      PID      PGID      STATE      COMMAND
-----
1        969      969      Done       sleep 10
-----

job-shell> cleanup
Completed jobs removed from job table.
job-shell> jobs

Job Table
-----
ID      PID      PGID      STATE      COMMAND
-----

job-shell> sleep 5 &
[2] PID=973 PGID=973 Running
job-shell> jobs
```

```
job-shell> jobs

Job Table
-----
ID      PID      PGID      STATE      COMMAND
-----

job-shell> sleep 5 &
[2] PID=973 PGID=973 Running
job-shell> jobs

Job Table
-----
ID      PID      PGID      STATE      COMMAND
-----
2        973      973      Done       sleep 5
-----

job-shell> jobs

Job Table
-----
ID      PID      PGID      STATE      COMMAND
-----
2        973      973      Done       sleep 5
-----

job-shell> cleanup
Completed jobs removed from job table.
job-shell> exit
Job table manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_16$ |
```